Chapter 4- Communication Interfacing Protocol

Synchronous Transmission

Synchronous transmission is a communication method in which data is transmitted continuously in a stream at a constant rate. In this method, the transmitting and receiving devices use synchronized clocks, meaning both operate at the same clock rate so the receiver can correctly interpret incoming bits.

Key Characteristics

1. Clock Synchronization
In synchronous communication, the clocks of the sender and receiver are synchronized. This synchronization ensures that the receiver reads each bit at the correct time during transmission.

2. Continuous Data Flow
Data is transmitted continuously without start and stop bits for each character. Because the devices remain synchronized, the receiver can continuously read the incoming data stream.

3. Block or Frame-Based Transmission
Data is sent in blocks called frames or packets instead of individual characters. Each frame contains multiple bytes of data, which improves transmission efficiency.

4. Synchronization Characters (SYN)
Before sending actual data, the transmitter sends special synchronization characters called SYN (synchronous characters).
These characters help the receiver synchronize its timing with the sender and prepare for the incoming data frame.

5. Synchronization Process

1. The sender transmits SYN characters.
2. The receiver detects and decodes these characters.
3. The receiver synchronizes its clock with the transmitter.
4. Once synchronization is established, data transmission begins.

6. Data Block Size
Data is transmitted in blocks containing multiple bytes. For example, a block may contain 164 bytes of data, which equals:

- 164 bytes × 8 bits = 1312 bits

7. Efficiency and Reliability
Synchronous transmission is efficient and reliable because:

- Data is sent continuously.
- Overhead is reduced since start and stop bits are not required for every character.
- It is suitable for transferring large amounts of data.

8. Duplex Communication
Synchronous systems can support full-duplex communication, allowing simultaneous sending and receiving of data between devices.

Protocols Using Synchronous Transmission

Many network communication technologies use synchronous transmission, such as:

- Ethernet
- SONET
- Token Ring

Examples of Applications

Applications that rely on communication systems using synchronous transmission include:

- Chat rooms
- Video conferencing / video calling
- Telephone conversations

Asynchronous Transmission

Asynchronous transmission is a communication method in which data is sent one character at a time rather than as a continuous stream. Each character is transmitted separately, and special bits are added to indicate the beginning and end of each character.

Key Characteristics

1. Start and Stop Bits
In asynchronous transmission, each data character is surrounded by **extra control bits**:

- A **start bit (0)** is placed before the data.
- One or more **stop bits (1)** are placed after the data.

These bits allow the receiver to identify **where each character begins and ends**.

2. Character-Based Transmission
Data is transmitted **one byte (one character) at a time** rather than in large blocks. A typical character consists of **8 data bits**.

Example of transmitted bits:

- Start bit (1 bit)
- Data bits (8 bits)
- Stop bit (1 or sometimes 2 bits)

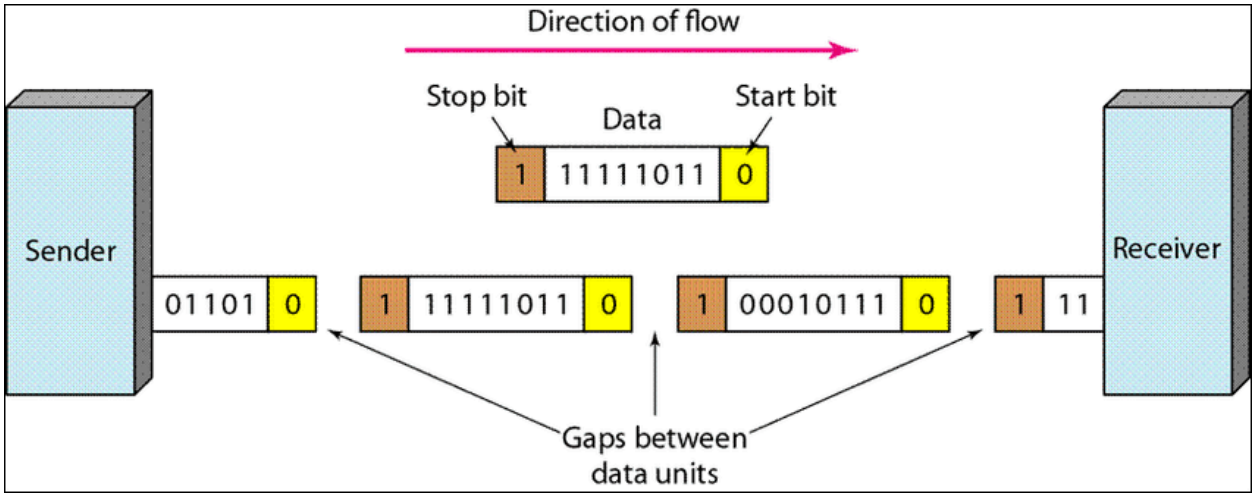
So, to send **8 bits of actual data**, the system usually sends **10 or sometimes 11 bits** in total.

3. Receiver Operation

When data arrives:

1. The receiver detects the **start bit (0)**.
2. The receiver then reads the **next 8 bits as the data character**.
3. After the data bits, the receiver checks the **stop bit (1)** to confirm that the character transmission is complete.

The start and stop bits provide **timing information and synchronization for each individual character**.



4. Half-Duplex Communication

Asynchronous transmission systems commonly operate in **half-duplex mode**, meaning communication occurs **in one direction at a time**.

5. Mark and Space States

During asynchronous communication, the line has two important states:

- **Mark state:**
  - Represents **binary 1** (or negative voltage).
  - Occurs when the line is **idle or no data is being transmitted**.
- **Space state:**
  - Represents **binary 0** (or positive voltage).

When the line changes from **mark (1)** to **space (0)**, the receiver recognizes that a **start bit** has arrived and that a data character will follow.

Because of this:

- The start bit is always 0 (space)
- The stop bit is always 1 (mark)

6. Data Transmission Pattern
Data in asynchronous transmission is sent **occasionally or at irregular intervals** rather than continuously. This makes it suitable for systems where data is generated from time to time.

7. Overhead and Speed
Because additional bits (start and stop bits) are added to every character, asynchronous transmission has **extra overhead**.
This overhead **reduces transmission efficiency and speed** compared to synchronous transmission.

8. Simplicity and Cost
Asynchronous transmission is:

- Simple to implement
- Less expensive
- Suitable for transmitting small amounts of data

Examples of Applications

Examples of systems that may use asynchronous communication include:

- Letters
- Radios
- Email
- Televisions

Difference between SYNCHRONOUS and ASYNCHRONOUS

BASIS FOR COMPARISON	SYNCHRONOUS TRANSMISSION	ASYNCHRONOUS TRANSMISSION
Meaning	Transmission starts with the block header which holds a sequence of bits.	It uses start bit and stop bit preceding and following a character respectively.
Transmission manner	Sends data in the form of blocks or frames (Full Duplex)	Sends 1 byte or character at a time (Half Duplex)
Synchronization	Present with the same clock pulse.	Absent
Transmission Speed	Fast	Slow
Gap between the data	Does not exist	Exist
Cost	Expensive	Economical
Time Interval	Constant	Random

Communication Protocols

Serial Peripheral Interface (SPI)

SPI (Serial Peripheral Interface) is a synchronous serial communication protocol used for fast and simple data transfer between devices, typically between a microcontroller (master) and peripheral devices (slaves).

Basic Characteristics

1. Master-Slave Architecture

SPI uses a Master-Slave communication model.

- There is **one master device**.
- There can be **multiple slave devices**.
- The **master controls the communication**.

The master:

- Generates the **clock signal**
- Selects the slave device
- Initiates data transfer

Slave devices **cannot control the clock**.

2. Synchronous Transmission

SPI is a synchronous protocol, meaning data transfer occurs in synchronization with a clock signal.

- Data is transferred only when the clock signal is present.
- The master generates the clock signal **(SCLK)**.

3. Full Duplex Communication

SPI supports full-duplex communication, meaning:

- Data can be **sent and received at the same time**.
- This is possible because separate lines are used for transmitting and receiving.

4. Data Rate

SPI supports **high data transfer speeds**, typically up to **around 10 Mbps**, depending on the devices used.

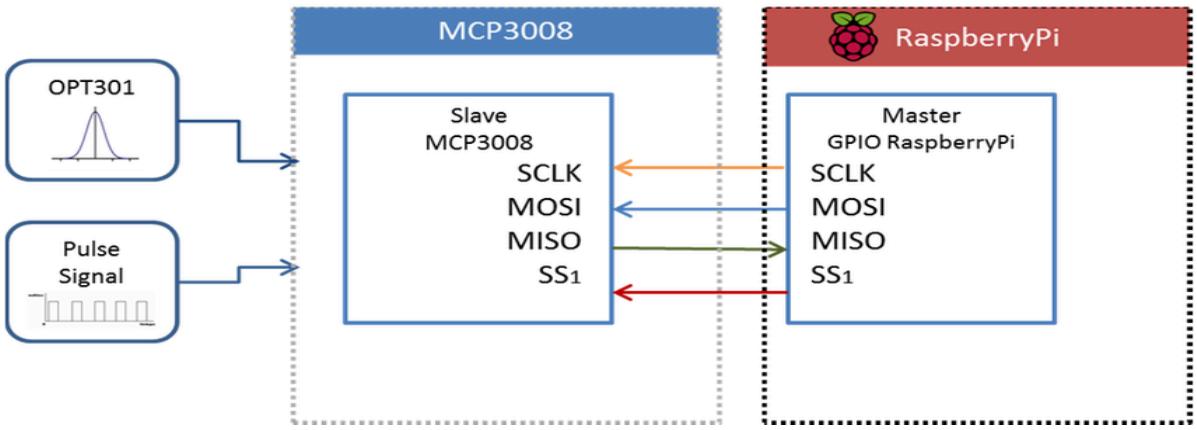
5. Power Consumption

SPI generally uses **lower power than I²C** because it **does not require pull-up resistors** on the communication lines.

SPI Signal Lines

A simple SPI interface uses **four signal wires**:

1. **MOSI (Master Out Slave In)**
   - Carries data **from the master to the slave**.
2. **MISO (Master In Slave Out)**
   - Carries data **from the slave to the master**.
3. **SCLK (Serial Clock)**
   - Clock signal generated by the **master** to synchronize data transmission.
4. **SS (Slave Select) or CS (Chip Select)**
   - Used by the **master to select which slave device to communicate with**.



Operation of Slave Select (SS)

- When the master wants to communicate with a slave device, it **sets the SS line of that slave to LOW (0)**.
- This **activates the slave**.
- If the SS line is **HIGH (1)**, the slave remains inactive.

In a system with multiple slaves:

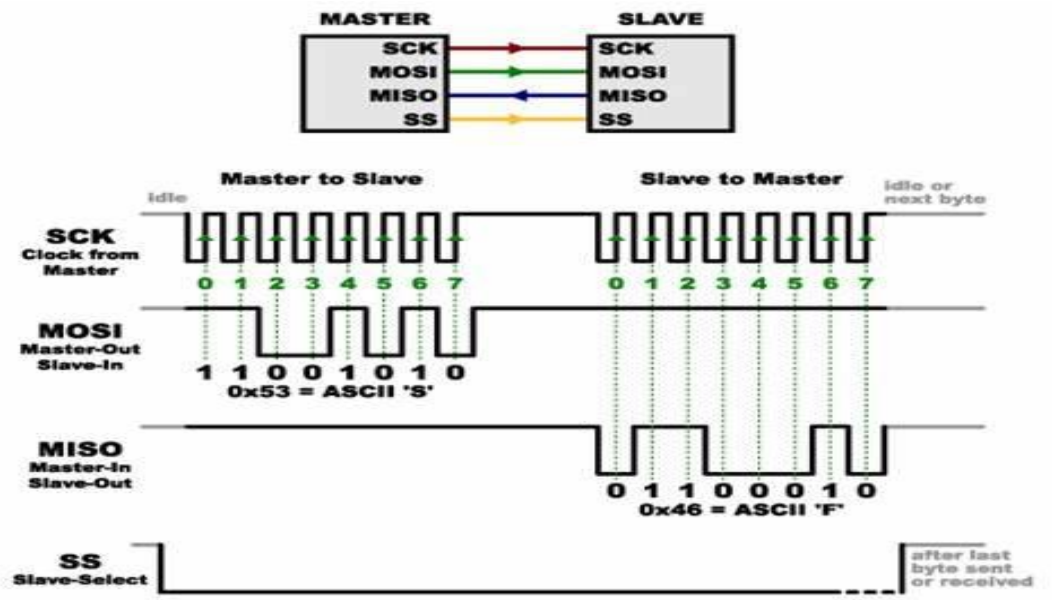
- Each slave usually has its **own SS line**.
- The slave whose **SS = 0** becomes active.

How SPI Communication Works

1. The master generates the clock signal (SCLK).
2. The master sets the SS/CS pin LOW to activate a specific slave.
3. The master sends data one bit at a time through MOSI.
4. The slave reads the incoming bits.
5. If the slave needs to respond, it sends data through MISO.
6. The master reads the incoming bits from the MISO line.

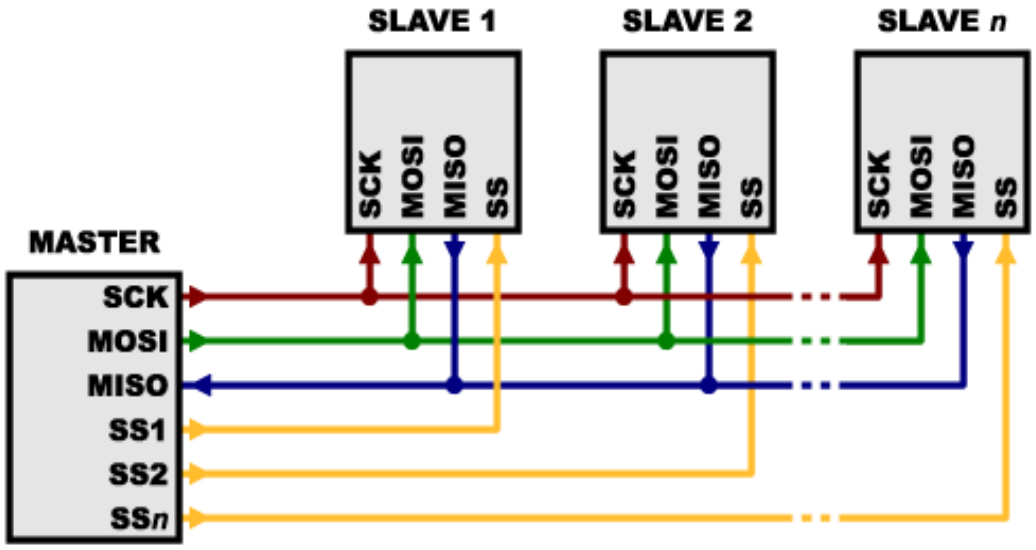
Thus:

- **MOSI → Master sends data to slave**
- **MISO → Slave sends data to master**



Multiple Slave Configuration

Single Master – Multiple Slaves

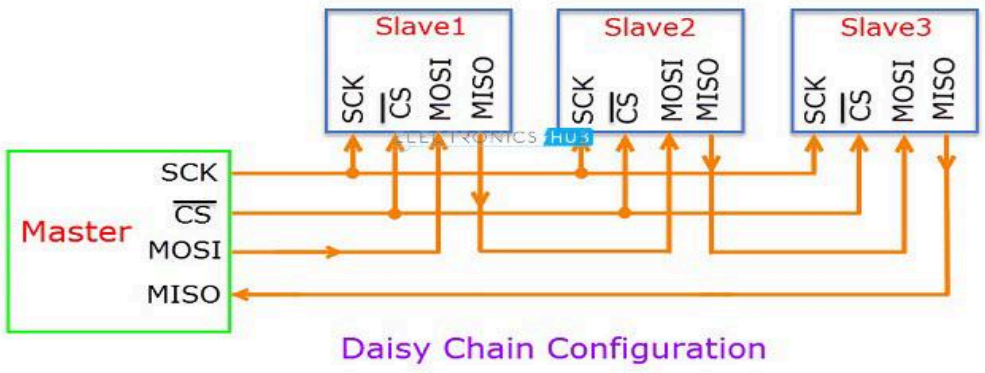


In this configuration:

- The master communicates with several slaves.
- Each slave has a separate SS line.
- The master activates a specific slave by setting its **SS line to 0**.

Drawback:
The number of required **SS pins increases with the number of slaves**, which makes the circuit more complex.

Daisy Chain Configuration



In a **daisy-chain SPI configuration**:

- Devices are connected **in series**.
- All devices share the **same SS line**.

Connection method:

- Master MOSI → First slave MOSI
- First slave MISO → Second slave MOSI
- Second slave MISO → Next slave MOSI
- Last slave MISO → Master MISO

In this arrangement:

- Data received by one slave is **passed to the next slave**.
- Often used when **the same type of device receives continuous data**.

Limitation:
Although wiring becomes simpler, **data takes longer to reach the final device**.

Advantages of SPI

- Full-duplex communication
- Separate MOSI and MISO lines, allowing simultaneous transmission and reception
- Continuous data streaming (no start and stop bits)
- Higher data transfer rate than I²C
- Flexible data size (not limited to 8-bit words)
- Flexible message size and format
- Low power consumption
- Simple protocol with no complex addressing system

Disadvantages of SPI

- Requires more pins than I²C
- No hardware flow control
- No slave acknowledgement mechanism
- Multi-master implementation is difficult
- No built-in error checking like the parity bit used in UART
- Usually requires separate SS lines for each slave, which can be problematic when many slaves are used

SPI Peripherals

SPI is commonly used to communicate with many types of peripheral devices, including:

Converters

- ADC (Analog-to-Digital Converter)
- DAC (Digital-to-Analog Converter)

Memory Devices

- EEPROM
- RAM
- Flash memory

Sensors

- Temperature sensors
- Humidity sensors
- Pressure sensors

Other Devices

- Real-time clocks
- Digital potentiometers
- LCD controllers
- UART controllers
- USB controllers
- CAN controllers
- Amplifiers

I²C Bus (Inter-Integrated Circuit)

I²C (Inter-Integrated Circuit) is a serial communication protocol used to connect multiple integrated circuits (ICs) using only two communication lines. It is commonly used for communication between microcontrollers and peripheral devices on the same board, and it can also connect components through cables.

The protocol is known for its **simplicity, flexibility, and ability to connect many devices using minimal wiring**.

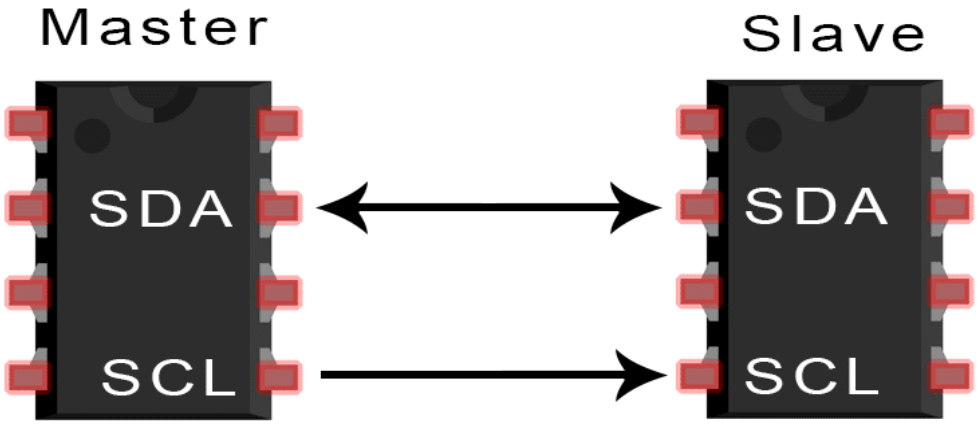
Main Features of I²C

1. Two Communication Lines

I²C requires only **two bus lines**:

- **SDA (Serial Data Line)** – used to transfer data between devices (always bidirectional)
- **SCL (Serial Clock Line)** – used to synchronize the data transfer (mostly bidirectional. mainly master-controlled, but not strictly one-way)

Both lines are **bidirectional** and shared by all devices connected to the bus.



2. Clock Generation

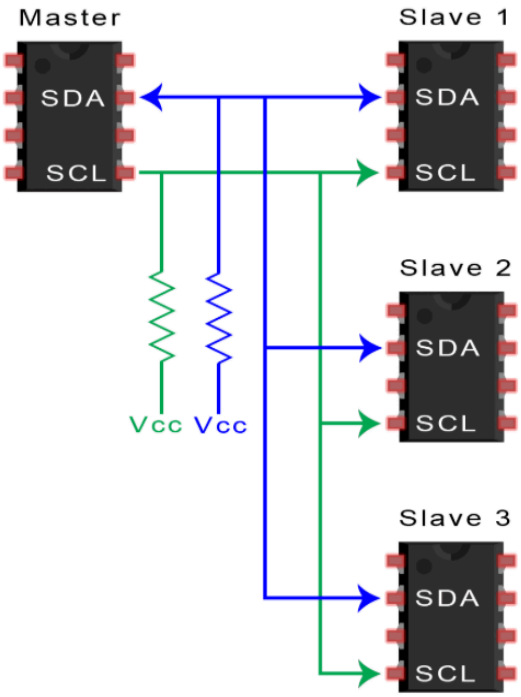
The **master device generates the clock signal on the SCL line**.
Therefore, I²C does not require a fixed baud rate like serial communication systems such as RS-232.

3. Master-Slave Communication

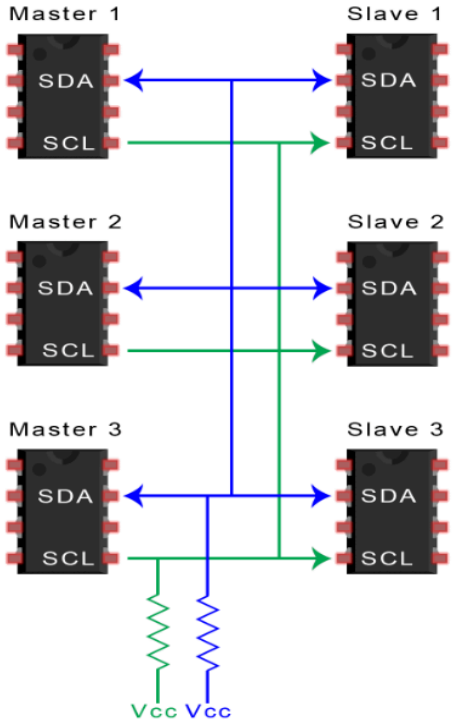
I²C devices communicate using a **master-slave relationship**:

- The **master device initiates communication** and controls the clock.
- Slave devices **respond when their address is called**.

I²C supports **multi-master systems**, meaning more than one master device can control the bus.



Single Master – Multiple Slave



Multiple Master – Multiple Slave

4. Device Addressing

Each device connected to the I²C bus has a **unique address**.

Two addressing formats exist:

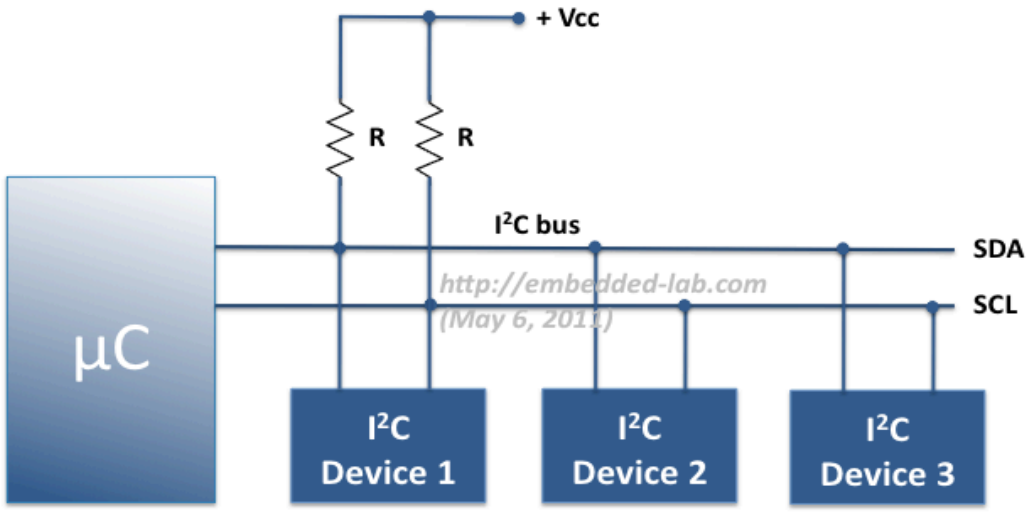
- **7-bit address**
- **10-bit address**

With a **7-bit address**, theoretically $\mathbf{2^7 = 128}$ **devices** can be addressed on the bus.

5. Arbitration and Collision Detection

I²C supports **multi-master operation**, meaning multiple masters can attempt communication.
The protocol includes **arbitration and collision detection**, allowing one master to gain control of the bus if multiple masters attempt communication at the same time.

I²C Protocol Basics



Multiple devices on common I²C bus

I²C communication is divided into two types of frames:

1. Address Frame

The **address frame is transmitted first** in every communication sequence.

- Contains the **7-bit or 10-bit slave address**
- Followed by a **Read/Write (R/W) bit**

R/W bit meaning:

- **0 → Write operation** (master sends data to slave)

- **1 → Read operation** (master receives data from slave)

The address bits are transmitted **Most Significant Bit (MSB) first**.

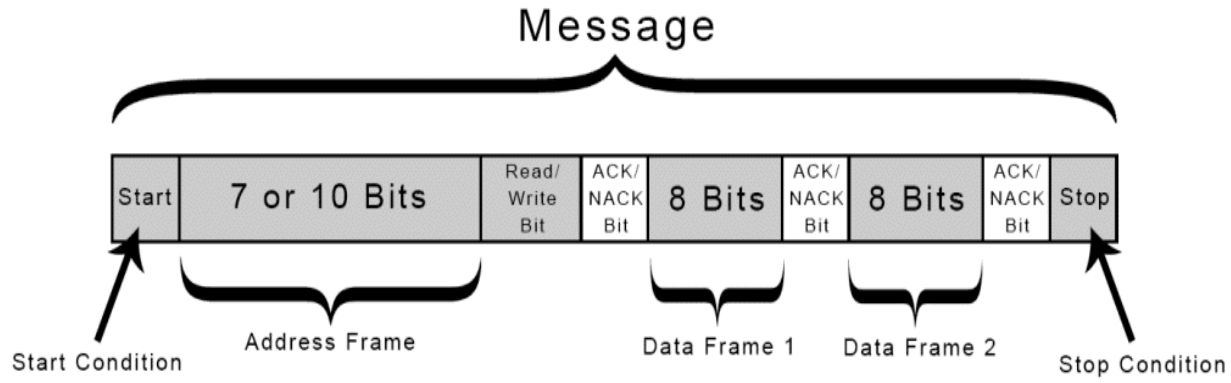
2. Data Frame

After the address frame, **data frames** are transmitted.

- Each data frame contains **8 bits of data**
- Data may be sent **from master to slave or from slave to master**

Data transmission rule:

- Data is **placed on SDA when SCL is low**
- Data is **read or sampled when SCL goes high**



Start Condition

Communication begins with a **Start Condition**.

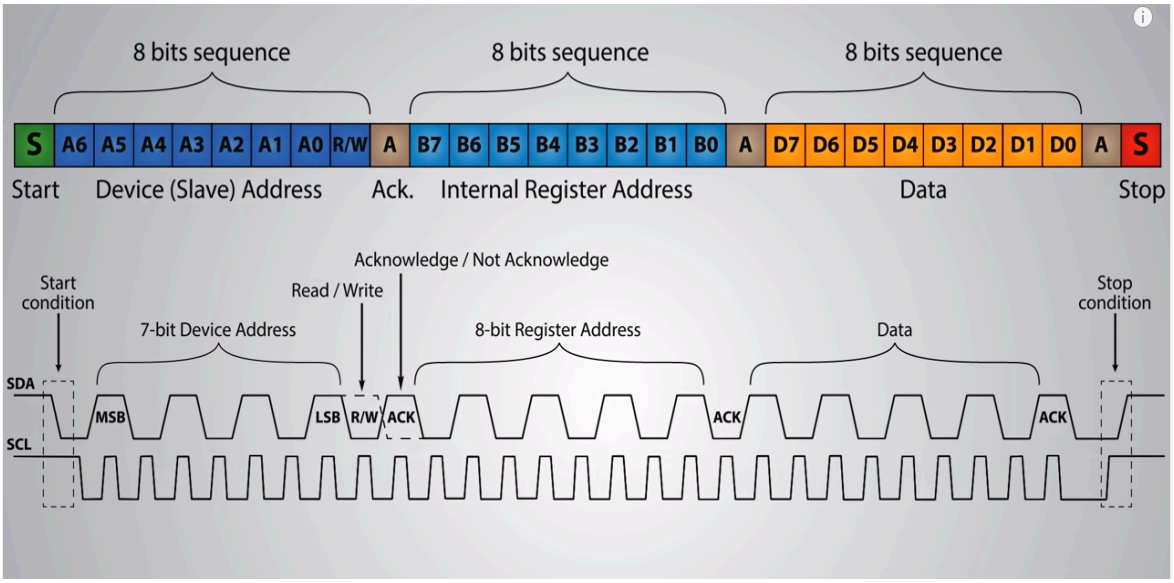
The master:

1. Keeps **SCL high**
2. Pulls **SDA from high to low**

This signals all slave devices that **communication is about to begin**.

If two masters attempt to start communication simultaneously, the device that **pulls SDA low first gains control of the bus**.

The master can also issue a **Repeated Start**, allowing another communication sequence **without releasing the bus**.



Idle State of the Bus

When there is **no communication**:

- **SDA = High**
- **SCL = High**

This condition indicates that the **bus is free**.

Acknowledgement Bit (ACK / NACK)

Each frame contains **9 bits**:

- 8 bits of data
- 1 acknowledgement bit

After the first 8 bits are sent:

- The **receiver takes control of the SDA line**.
- If the receiver **pulls SDA low**, it sends an **ACK (Acknowledgement)**.
- If SDA remains **high**, it sends a **NACK (No Acknowledgement)**.

If a NACK occurs, the communication may **stop or be retried by the master**.

Data Frames

After the address frame:

- The master continues generating **clock pulses on SCL**.
- Data bits are placed on the **SDA line**.
- Depending on the **R/W bit**, either the **master or the slave sends data**.
- Each data frame is followed by an **ACK/NACK bit**.

Stop Condition

Communication ends with a **Stop Condition**.

The master generates the stop condition by:

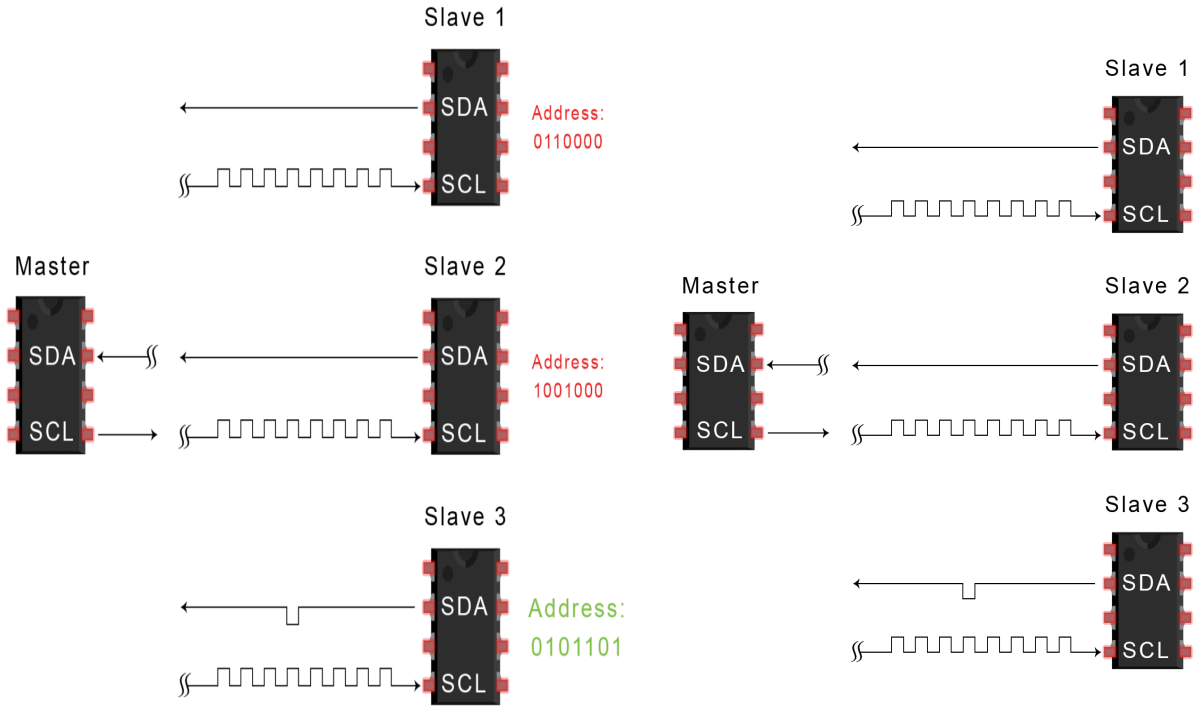
1. Making **SCL high**
2. Changing **SDA from low to high**

During normal data transmission, **SDA should not change when SCL is high**, otherwise it may be interpreted as a start or stop condition.

Steps of I²C Data Transmission

1. The master sends a **Start Condition** by pulling SDA low while SCL is high.
2. The master sends the **7-bit or 10-bit address** of the target slave along with the **R/W bit**.
3. Each slave compares the received address with its own address.
4. The matching slave sends an **ACK bit by pulling SDA low**.
5. Data frames are transferred between master and slave.
6. After each data frame, the receiver sends an **ACK bit**.

7. When communication finishes, the master generates a **Stop Condition**.



Advantages of I²C

1. **Flexibility** – Supports **multi-master and multi-slave communication**.
2. **Addressing Feature** – Devices are selected by **unique addresses**, eliminating the need for separate slave select lines.
3. **Simplicity** – Only **two signal lines** are required for communication.
4. **Error Handling** – The **ACK/NACK mechanism** helps detect transmission errors.
5. **Adaptability** – Can work with both **slow and fast devices**.

Disadvantages of I²C

1. **Address Conflicts** – Devices must have **unique addresses**, which can sometimes cause conflicts.
2. **Lower Speed** – Data transfer speed is **slower than SPI**.

3. **More Complex Hardware** – Requires additional components such as **pull-up resistors**.
4. **Data Frame Size** – Data is transferred in **8-bit frames**, which can limit efficiency for large data transfers.

UART (Universal Asynchronous Receiver/Transmitter)

UART (Universal Asynchronous Receiver/Transmitter) is a hardware communication protocol used for serial data transmission between two devices.
It is also known as SCI (Serial Communication Interface).

UART provides:

- Full-duplex communication
- Asynchronous communication
- Simple and low-cost device communication

It is commonly used to connect peripherals to microcontrollers and computers.

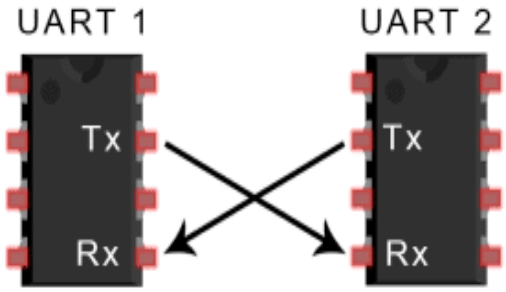
Example applications include connecting:

- GPS modules
- Bluetooth modules
- RFID card readers

to microcontrollers such as **Arduino** or **Raspberry Pi**.

Main Characteristics

1. Serial Communication
UART's main purpose is to **transmit and receive serial data**. Data is sent **one bit at a time over a communication line**.



2. Two-Wire Communication

UART typically uses **two signal lines**:

- **TX (Transmit)** – sends data
- **RX (Receive)** – receives data

Data flows from the **TX pin of the transmitting device** to the **RX pin of the receiving device**.

A **ground wire** is also used as a common reference.

3. Full-Duplex Communication

UART supports **full-duplex communication**, meaning:

- Devices can **send and receive data simultaneously**.

4. Asynchronous Communication

UART is an **asynchronous protocol**, meaning:

- **No clock signal is shared between devices**.
- Both devices must be configured to operate at the **same baud rate**.

5. Standard Baud Rates

The **baud rate** is the speed of data transmission measured in **bits per second (bps)**.

Common UART baud rates include:

100, 200, 300, 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200 bps

Both communicating devices must use **approximately the same baud rate**.
A difference of about **10% or less** is generally acceptable.

Why Use UART?

UART is commonly used when:

- Very high speed is not required
- A simple and inexpensive communication link is needed between two devices

UART communication is inexpensive because:

- Only one wire is required for each direction
- Simple hardware implementation

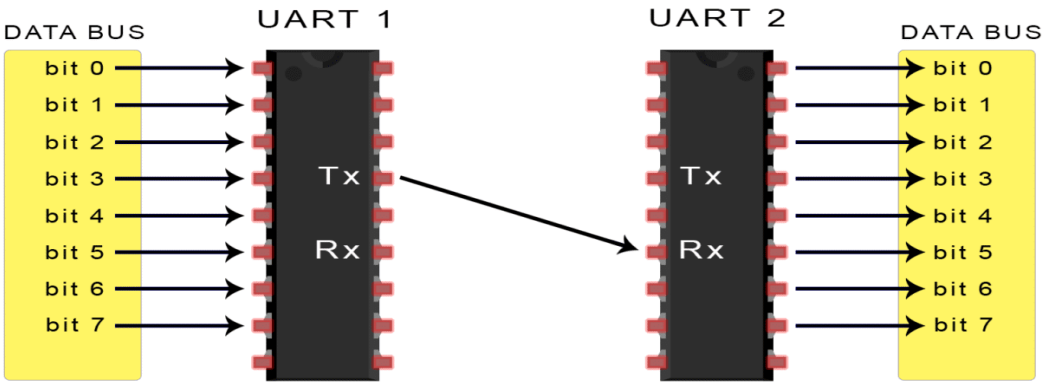
How UART Works

Two UART devices communicate **directly with each other**.

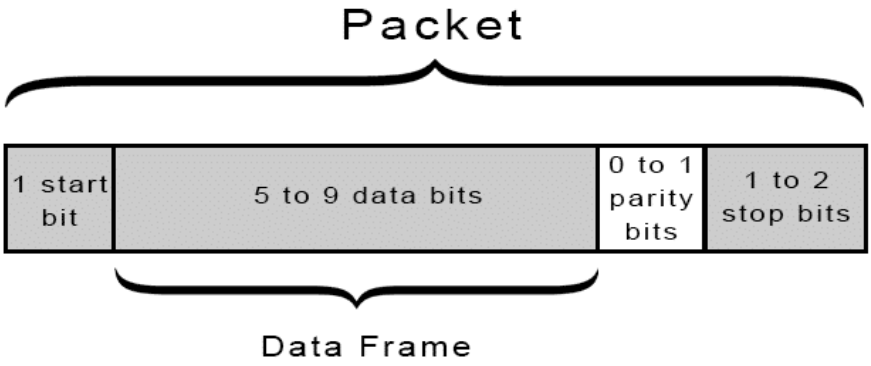
1. Data from a device such as a **CPU or microcontroller** is sent to the UART through a **data bus in parallel form**.
2. The transmitting UART **converts the parallel data into serial form**.
3. The UART adds **control bits** (start, parity, stop).
4. The data packet is sent **bit by bit through the TX pin**.
5. The receiving UART reads the bits through its **RX pin**.
6. The receiving UART **removes the start, parity, and stop bits**.
7. The data is converted **back to parallel form** and sent to the receiving device through the data bus.

Data flow:

Data Bus → Parallel Data → Transmitting UART → Serial Data → Receiving UART → Parallel Data → Data Bus



UART Data Frame



UART communication uses a **data frame structure** containing several bits.

1. Start Bit

The transmission line normally stays **high when idle**.

To start transmission:

- The transmitting UART **pulls the line from high to low**.
- This **high-to-low transition signals the start of data transmission**.

2. Data Bits

The **data frame contains the actual data**.

- Typically **5 to 8 data bits**
- In some configurations **up to 9 bits**
- Data is usually sent **Least Significant Bit (LSB) first**

3. Parity Bit (Optional)

The **parity bit** is used for **error detection**.

Two types of parity exist:

- **Even parity** – total number of 1 bits should be even (No error)
- **Odd parity** – total number of 1 bits should be odd (Error)

The receiving UART checks the parity to determine if **any bit errors occurred during transmission**.

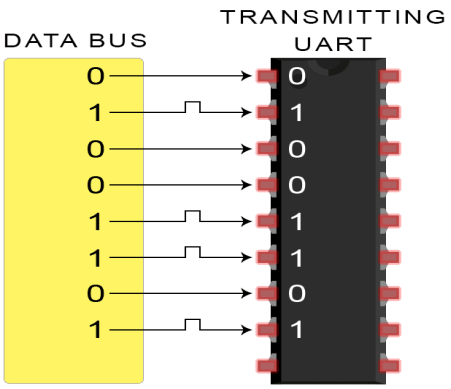
4. Stop Bits

Stop bits signal the **end of the data packet**.

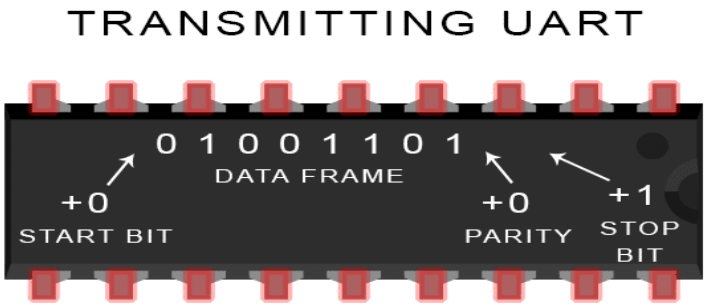
- The transmission line returns to **high voltage**
- Usually **1 or 2 stop bits** are used

Steps of UART Transmission

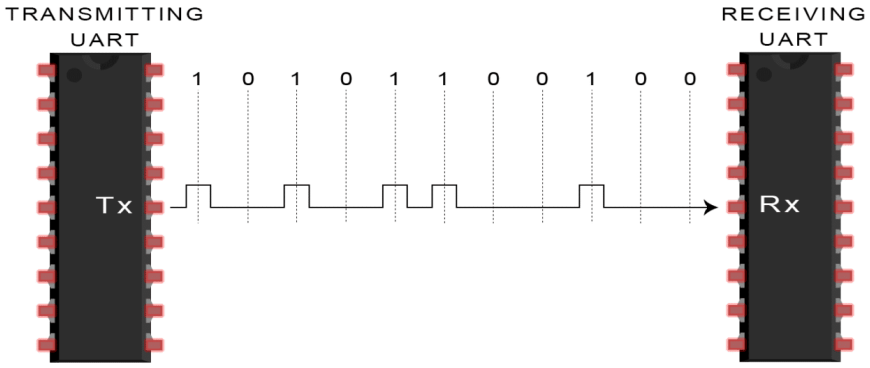
1. The transmitting UART receives **parallel data from the data bus**.



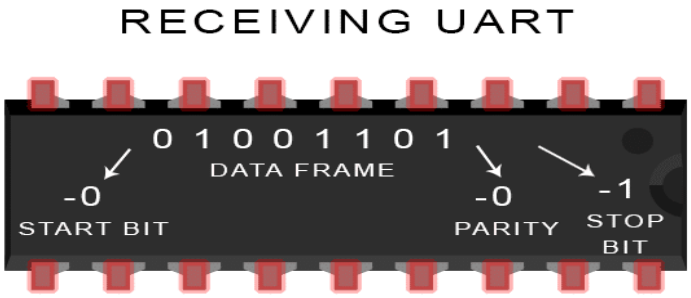
2. The UART **adds start, parity, and stop bits** to form a data frame.



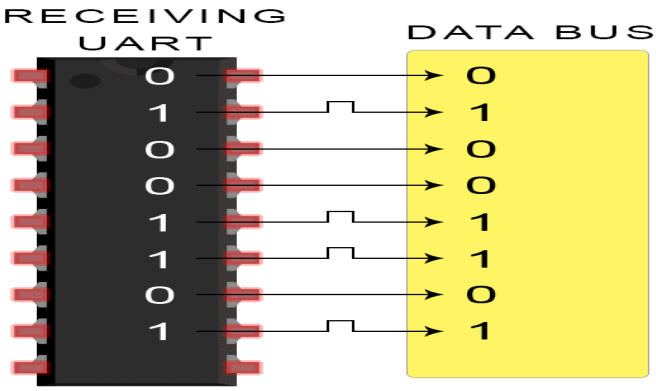
3. The data frame is **sent serially through the TX line**.
4. The receiving UART **samples the data using the configured baud rate**.



5. The receiver **removes the start, parity, and stop bits**.



6. The UART converts the serial data **back into parallel form**.



7. The data is sent to the receiving device through the **data bus**.

Advantages of UART

- Uses only two communication wires (TX and RX)
- No clock signal required
- Includes parity bit for error detection
- Flexible data packet structure
- Simple and inexpensive hardware
- Widely used and well documented

Disadvantages of UART

- Data frame size is limited (maximum about 9 data bits)
- Lower data speed compared to SPI
- No built-in addressing system
- Supports communication only between two devices
- Does not support multi-master or multi-slave systems